Universidad Cenfotec

Portafolio #2

Curso: Programación orientada a objetos

Estudiante: Fabiola Román Barrantes

Profesor: Romario Salas Cerdas

Fecha: 26-04-2026

Tabla de Contenidos

1. INTRODUCCIÓN 3
2. DESCRIPCIÓN DEL SISTEMA 3
3. DISEÑO ORIENTADO A OBJETOS 4
4. APLICACIÓN DE LA PROGRAMACIÓN ORIENTADA A OBJETOS 4
5. ARQUITECTURA DEL SISTEMA (MVC) 5
6. IMPLEMENTACIÓN DEL PATRÓN DAO 5
7. PERSISTENCIA DE DATOS 5
8. MANEJO DE EXCEPCIONES 6
9. DIAGRAMA UML 6
10. REPOSITORIO GITHUB 8
11. REFLEXIÓN PERSONAL 8
12. CONCLUSIÓN 8

1. INTRODUCCIÓN

La programación orientada a objetos (POO) es un paradigma fundamental en el desarrollo de software, ya que permite modelar sistemas del mundo real mediante el uso de clases y objetos. Este enfoque facilita la organización del código, promueve la reutilización y mejora la mantenibilidad de los sistemas.

En este portafolio se presenta el desarrollo de un sistema de gestión de biblioteca, en el cual se aplican los principales conceptos de la programación orientada a objetos, tales como la abstracción, el encapsulamiento y las relaciones entre clases. Asimismo, se incorporan elementos más avanzados como la arquitectura Modelo-Vista-Controlador (MVC), el patrón de diseño DAO, el manejo de excepciones y la persistencia de datos mediante archivos.

El propósito de este trabajo es evidenciar la comprensión y aplicación práctica de los conceptos estudiados durante el curso en un sistema funcional y estructurado.

2. DESCRIPCIÓN DEL SISTEMA

El sistema desarrollado corresponde a una aplicación de consola que permite gestionar una biblioteca de forma básica. Entre sus principales funcionalidades se encuentran:

- Registro de libros
- Visualización de libros disponibles
- Búsqueda de libros por título
- Préstamo de libros a usuarios

El sistema modela entidades del mundo real como libros, usuarios y préstamos, permitiendo representar de manera sencilla las interacciones que ocurren dentro de una biblioteca.

Este tipo de sistema fue seleccionado debido a que facilita la aplicación de los principios de la programación orientada a objetos y permite establecer relaciones claras entre las diferentes clases.

3. DISEÑO ORIENTADO A OBJETOS

Para el desarrollo del sistema se identificaron las principales entidades del problema y se representaron mediante clases.

Las clases implementadas son:

Libro: representa un libro dentro del sistema, con atributos como título, autor y disponibilidad.

Usuario: representa una persona que puede interactuar con el sistema y realizar préstamos.

Préstamo: establece la relación entre un usuario y un libro cuando este es prestado.

BibliotecaController: gestiona la lógica del sistema y coordina la interacción entre las diferentes clases.

LibroDAO: se encarga del acceso y manipulación de los datos almacenados.

Este diseño permite una adecuada separación de responsabilidades y una estructura organizada del sistema.

4. APLICACIÓN DE LA PROGRAMACIÓN ORIENTADA A OBJETOS

Abstracción

Se aplicó la abstracción al representar entidades del mundo real mediante clases, enfocándose en sus características esenciales.

Encapsulamiento

Se utilizaron atributos privados y métodos públicos para controlar el acceso a la información, garantizando la integridad de los datos.

Relaciones entre clases

Se implementaron diferentes tipos de relaciones:

- Asociación entre Usuario y Préstamo
- Composición entre Préstamo y Libro
- Dependencia entre BibliotecaController y LibroDAO

Estas relaciones permiten la interacción entre los objetos del sistema.

5. ARQUITECTURA DEL SISTEMA (MVC)

El sistema fue estructurado utilizando el patrón Modelo-Vista-Controlador (MVC), el cual permite separar responsabilidades.

Modelo:
Incluye las clases Libro, Usuario y Préstamo, que representan los datos del sistema.

Vista:
Representada por la clase Main, encargada de la interacción con el usuario mediante la consola.

Controlador:
La clase BibliotecaController gestiona la lógica del sistema y actúa como intermediario entre la vista y el modelo.

Esta arquitectura mejora la organización del código y facilita su mantenimiento.

6. IMPLEMENTACIÓN DEL PATRÓN DAO

Se implementó el patrón DAO mediante la clase LibroDAO, cuya función es gestionar el acceso a los datos.

Este patrón permite separar la lógica de negocio de la lógica de acceso a datos, lo que mejora la claridad del código y facilita posibles cambios futuros en la forma de almacenamiento de la información.

7. PERSISTENCIA DE DATOS

La persistencia de datos se implementó mediante el uso de archivos de texto (.txt).

Esto permite almacenar la información de los libros de manera permanente, evitando la pérdida de datos al finalizar la ejecución del programa.

Los datos se guardan en un formato estructurado que facilita su lectura y posterior procesamiento.

8. MANEJO DE EXCEPCIONES

Se implementó el manejo de excepciones mediante estructuras try-catch, con el objetivo de evitar errores en tiempo de ejecución.

Esto permite controlar situaciones como:

- Intento de préstamo de un libro no disponible
- Búsqueda de un libro inexistente

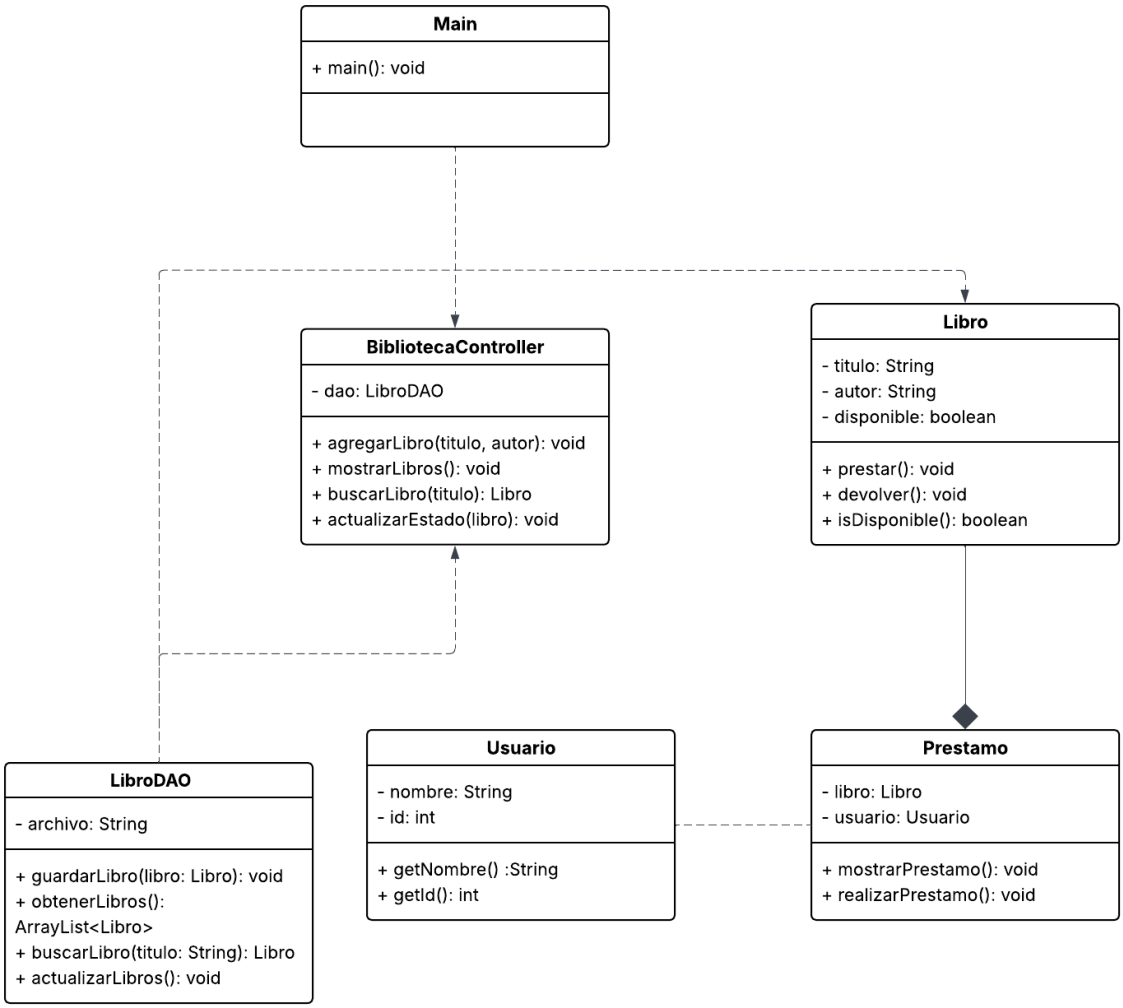
El uso de excepciones contribuye a la estabilidad y robustez del sistema.

9. DIAGRAMA UML

El diagrama UML representa la estructura del sistema y las relaciones entre las clases.

Incluye:

- Clases principales
- Atributos y métodos
- Relaciones entre clases


Main
+ main(): void
BibliotecaController
- dao: LibroDAO
+ agregarLibro(titulo, autor): void
+ mostrarLibros(): void
+ buscarLibro(titulo): Libro
+ actualizarEstado(libro): void
Libro
- titulo: String
- autor: String
- disponible: boolean
+ prestar(): void
+ devolver(): void
+ isDisponible(): boolean
LibroDAO
- archivo: String
+ guardarLibro(libro: Libro): void
+ obtenerLibros(): ArrayList<Libro>
+ buscarLibro(titulo: String): Libro
+ actualizarLibros(): void
Usuario
- nombre: String
- id: int
+ getNombre() :String
+ getId(): int
Prestamo
- libro: Libro
- usuario: Usuario
+ mostrarPrestamo(): void
+ realizarPrestamo(): void

10. REPOSITORIO GITHUB

El proyecto se encuentra almacenado en un repositorio de GitHub, lo que permite el control de versiones y una adecuada organización del código fuente.

11. REFLEXIÓN PERSONAL

Durante el desarrollo de este proyecto fue posible comprender de manera más profunda los conceptos de la programación orientada a objetos y su aplicación en un contexto real.

Uno de los principales aprendizajes fue la importancia de estructurar adecuadamente el sistema mediante el uso de arquitecturas como MVC y patrones como DAO, lo cual permite una mejor organización y mantenimiento del código.

Asimismo, la implementación de persistencia de datos y manejo de excepciones representó un reto significativo, ya que implicó considerar aspectos adicionales más allá de la lógica básica del programa.

Este proyecto representa un avance importante en el proceso de formación, ya que permitió consolidar conocimientos y desarrollar habilidades fundamentales para el desarrollo de software.

12. CONCLUSIÓN

El desarrollo de este portafolio permitió aplicar de manera práctica los conceptos fundamentales de la programación orientada a objetos.

Se logró construir un sistema funcional que demuestra la correcta implementación de principios de diseño, patrones de arquitectura y buenas prácticas de programación.

En conclusión, este trabajo evidencia la capacidad de integrar conocimientos teóricos en un proyecto real, fortaleciendo las competencias necesarias para el desarrollo de software de calidad.